

Medical Data History

(SQL Data Analysis Project)

- **Project Overview**
- **Database Schema**
- **Approach**
- **Queries (Explanation)**
- **Key Skills Demonstrated**
- **Conclusion**

Member: Ajith Kumar

Project ID: PRSQL - 02

TeamID: PTID-CDA-JUL-26-11181

1. Project Overview

This project involved analysing a relational healthcare dataset using SQL. The dataset was provided by the institute and accessed via a remote MySQL server through DBeaver. It consists of four related tables, patients, doctors, admissions, and province_names, covering patient demographics, hospital admission records, and attending physician details.

The objective was to answer 34 distinct business and analytical questions using SQL alone, covering filtering, aggregation, joins across multiple tables, string and date manipulation, and conditional logic, the core query skills used day-to-day in a data analyst role.

2. Database Schema

The database consists of four tables, related as follows:

- patients - patient_id (PK), first_name, last_name, gender, birth_date, city, allergies, height, weight, province_id (FK to province_names)
- doctors - doctor_id (PK), first_name, last_name, speciality
- admissions - patient_id (FK to patients), admission_date, discharge_date, diagnosis, doctor_id (FK to doctors)
- province_names - province_id (PK), province_name

Schema adjustment: the admissions table originally used the column name attending_doctor_id. This was renamed to doctor_id to align with the primary key in the doctors table, enabling a clean join between the two:

```
ALTER TABLE admissions CHANGE attending_doctor_id doctor_id INTEGER; (if permission given)
```

3. Approach

Questions were worked through in progressive groups rather than strict numerical order - starting with single-table filtering and sorting, then string/date functions, then joins across tables, and finally grouping and aggregation with HAVING. This built-up SQL fluency incrementally, with each group reinforcing concepts from the last.

Note on permissions: the provided database account had read-only access to the patients table. Question 5 (an UPDATE statement) could not be executed directly; a SELECT using COALESCE(allergies, 'NKA') was used to verify the intended logic non-destructively. The correct UPDATE syntax is documented below regardless, as understanding write operations and their WHERE-clause risk is part of the exercise.

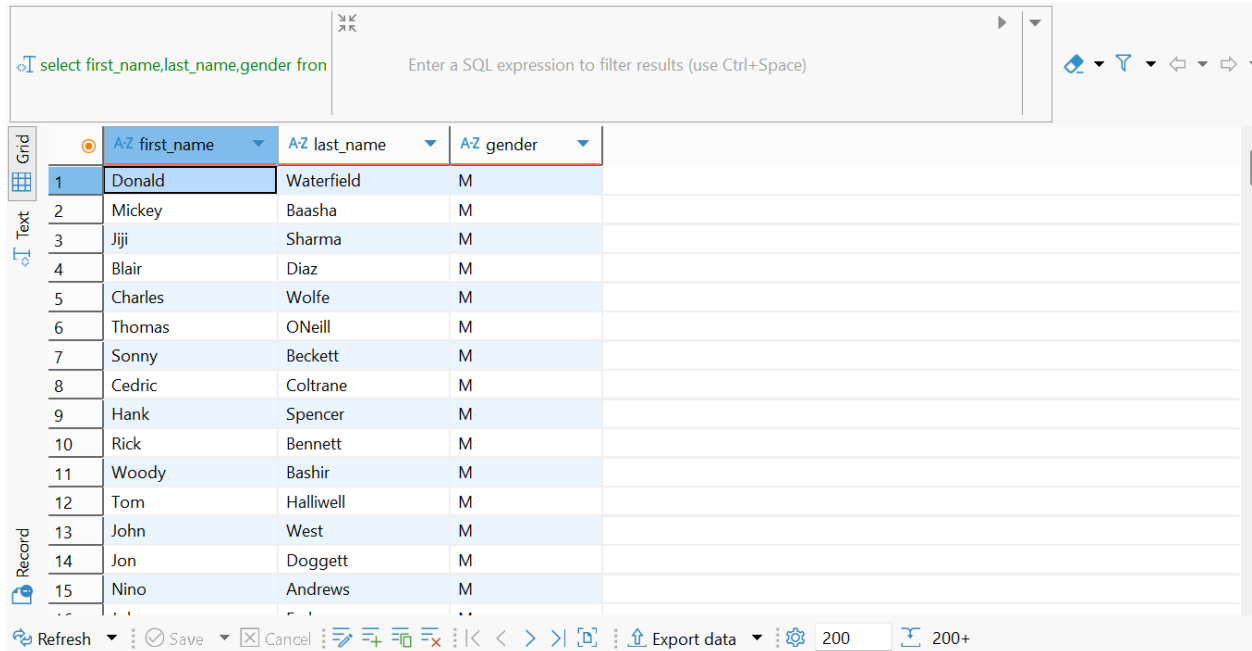
4. Queries and Explanations

Question 1

Show first name, last name, and gender of patients whose gender is 'M'.

```
SELECT first_name, last_name, gender FROM patients WHERE gender = 'M';
```

Basic SELECT with a WHERE filter, the two foundational building blocks of SQL querying.



The screenshot shows a database interface with a SQL query editor at the top and a results grid below. The query is `select first_name, last_name, gender from patients`. The results grid displays 15 rows of patient data, all with gender 'M'. The interface includes a 'Text' view toggle, a 'Record' view toggle, and a bottom toolbar with 'Refresh', 'Save', 'Cancel', and 'Export data' buttons.

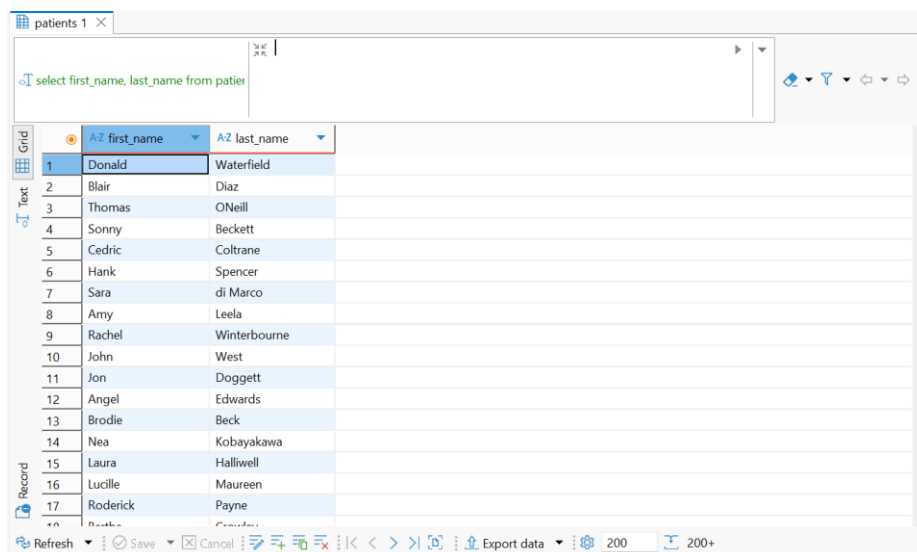
	A-Z first_name	A-Z last_name	A-Z gender
1	Donald	Waterfield	M
2	Mickey	Baasha	M
3	Jiji	Sharma	M
4	Blair	Diaz	M
5	Charles	Wolfe	M
6	Thomas	ONeill	M
7	Sonny	Beckett	M
8	Cedric	Coltrane	M
9	Hank	Spencer	M
10	Rick	Bennett	M
11	Woody	Bashir	M
12	Tom	Halliwell	M
13	John	West	M
14	Jon	Doggett	M
15	Nino	Andrews	M

Question 2

Show first name and last name of patients who do NOT have allergies.

```
SELECT first_name, last_name FROM patients WHERE allergies IS NULL;
```

Missing data is represented as NULL, not an empty string. NULL requires the special IS NULL / IS NOT NULL operators, it cannot be checked with '='.



The screenshot shows a database interface with a SQL query editor at the top and a results grid below. The query is `select first_name, last_name from patients`. The results grid displays 17 rows of patient data, all with NULL allergies. The interface includes a 'Text' view toggle, a 'Record' view toggle, and a bottom toolbar with 'Refresh', 'Save', 'Cancel', and 'Export data' buttons.

	A-Z first_name	A-Z last_name
1	Donald	Waterfield
2	Blair	Diaz
3	Thomas	ONeill
4	Sonny	Beckett
5	Cedric	Coltrane
6	Hank	Spencer
7	Sara	di Marco
8	Amy	Leela
9	Rachel	Winterbourne
10	John	West
11	Jon	Doggett
12	Angel	Edwards
13	Brodie	Beck
14	Nea	Kobayakawa
15	Laura	Halliwell
16	Lucille	Maureen
17	Roderick	Payne

Question 3

Show first name of patients that start with the letter 'C'.

```
SELECT first_name FROM patients WHERE first_name LIKE 'C%';
```

LIKE performs pattern matching; % is a wildcard meaning "any characters, any length."

The screenshot shows a database application window titled "patients 1". The SQL query entered is "select first_name from patients where fi". The results are displayed in a grid view with a single column "A-Z first_name". The first 17 rows of the grid contain the following names: Charles, Cedric, Charles, Cross, Calleigh, Catherine, Caroline, Casanova, Chen, Charmian, Cary, Christine, Carol, Corrado, Cordelia, Claire, and Carol. The interface includes a toolbar with icons for grid, text, and record views, as well as buttons for Refresh, Save, Cancel, and Export data.

	A-Z first_name
1	Charles
2	Cedric
3	Charles
4	Cross
5	Calleigh
6	Catherine
7	Caroline
8	Casanova
9	Chen
10	Charmian
11	Cary
12	Christine
13	Carol
14	Corrado
15	Cordelia
16	Claire
17	Carol

Question 4

Show first name and last name of patients that weigh within the range of 100 to 120 (inclusive).

```
SELECT first_name, last_name FROM patients WHERE weight BETWEEN 100 AND 120;
```

BETWEEN is inclusive on both ends; shorthand for weight >= 100 AND weight <= 120.

The screenshot shows a database application window titled "patients 1". The SQL query entered is "select first_name, last_name from patier". The results are displayed in a grid view with two columns: "A-Z first_name" and "A-Z last_name". The first 16 rows of the grid contain the following names: Jiji Sharma, Blair Diaz, Thomas ONeill, Sonny Beckett, Tom Halliwell, Jon Doggett, Angel Edwards, John Farley, Temple Russert, Don Edwards, Gary Ramotswe, Phil Chester, Woody Riviera, Rocky Donahue, Shiro Whale, and Didi Dean. The interface includes a toolbar with icons for grid, text, and record views, as well as buttons for Refresh, Save, Cancel, and Export data.

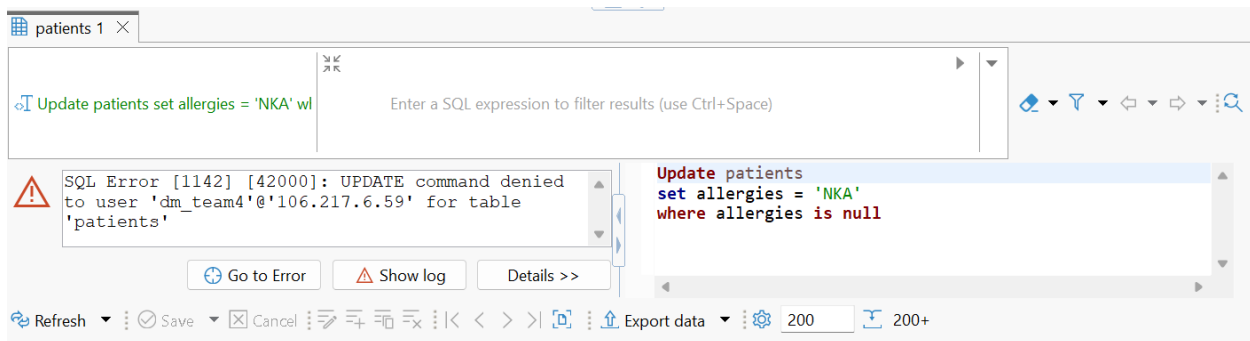
	A-Z first_name	A-Z last_name
1	Jiji	Sharma
2	Blair	Diaz
3	Thomas	ONeill
4	Sonny	Beckett
5	Tom	Halliwell
6	Jon	Doggett
7	Angel	Edwards
8	John	Farley
9	Temple	Russert
10	Don	Edwards
11	Gary	Ramotswe
12	Phil	Chester
13	Woody	Riviera
14	Rocky	Donahue
15	Shiro	Whale
16	Didi	Dean

Question 5

Update the patients table: if allergies is NULL, replace it with 'NKA'.

```
UPDATE patients SET allergies = 'NKA' WHERE allergies IS NULL;
```

UPDATE modifies existing data rather than reading it. The WHERE clause is critical — without it, every row would be overwritten. (Note: this account had read-only access to the patients table; COALESCE (allergies,'NKA') in a SELECT was used as a non-destructive equivalent for verification.)



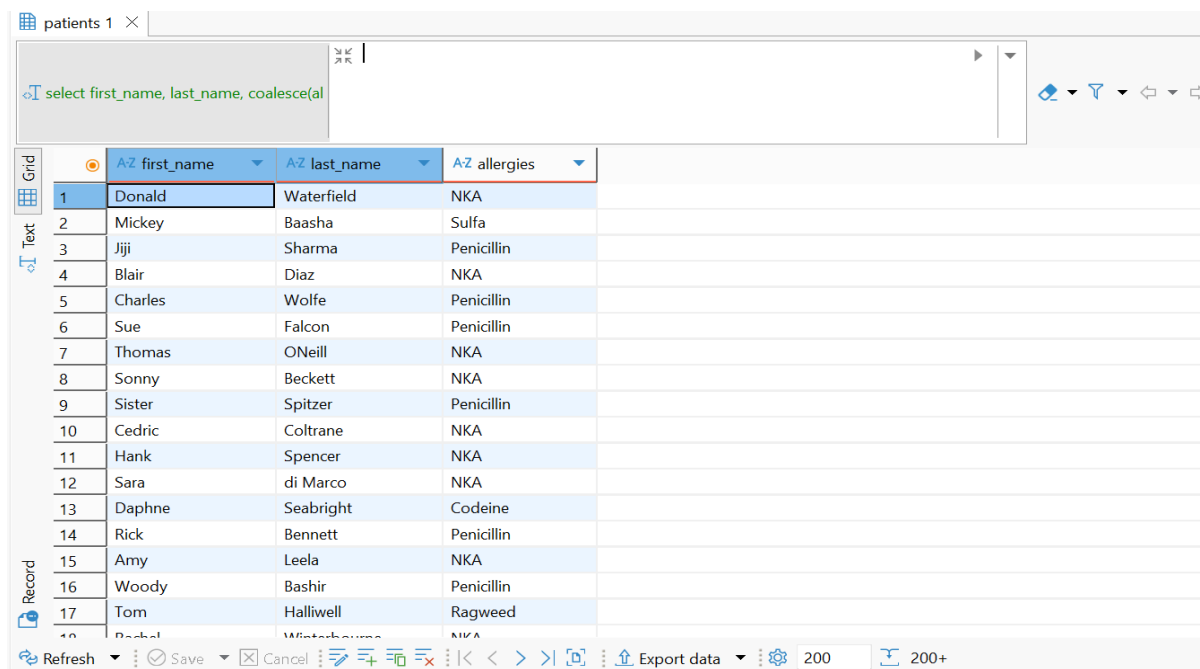
patients 1 X

Update patients set allergies = 'NKA' where allergies is null

SQL Error [1142] [42000]: UPDATE command denied to user 'dm_team4'@'106.217.6.59' for table 'patients'

Go to Error Show log Details >>

Refresh Save Cancel Export data 200 200+



patients 1 X

select first_name, last_name, coalesce(al

	A-Z first_name	A-Z last_name	A-Z allergies
1	Donald	Waterfield	NKA
2	Mickey	Baasha	Sulfa
3	Jiji	Sharma	Penicillin
4	Blair	Diaz	NKA
5	Charles	Wolfe	Penicillin
6	Sue	Falcon	Penicillin
7	Thomas	ONeill	NKA
8	Sonny	Beckett	NKA
9	Sister	Spitzer	Penicillin
10	Cedric	Coltrane	NKA
11	Hank	Spencer	NKA
12	Sara	di Marco	NKA
13	Daphne	Seabright	Codeine
14	Rick	Bennett	Penicillin
15	Amy	Leela	NKA
16	Woody	Bashir	Penicillin
17	Tom	Halliwell	Ragweed

Refresh Save Cancel Export data 200 200+

Question 6

Show first name and last name concatenated into one column as full name.

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM patients;
```

CONCAT() joins strings together; AS renames the output column for readability.

patients 1

```
select concat (first_name, ' ', last_name)
```

Grid

	AZ full_name
1	Donald Waterfield
2	Mickey Baasha
3	Jiji Sharma
4	Blair Diaz
5	Charles Wolfe
6	Sue Falcon
7	Thomas ONeill
8	Sonny Beckett
9	Sister Spitzer
10	Cedric Coltrane
11	Hank Spencer
12	Sara di Marco
13	Daphne Seabright
14	Rick Bennett
15	Amy Leela
16	Woody Bashir

Text

Record

Column: last_name varchar(150)
Table: project_medical_data_history.patients
Read-only: No unique key. Row modification is not available.

Refresh Save Cancel Export data 200 200+

Question 7

Show first name, last name, and the full province name of each patient.

```
SELECT p.first_name, p.last_name, pn.province_name FROM patients p JOIN
province_names pn ON p.province_id = pn.province_id;
```

A JOIN across two tables, matched on the shared province_id key. Table aliases (p, pn) shorten repeated references.

patients(+) 1

```
select p.first_name, p.last_name, pn.pro
```

Grid

	AZ first_name	AZ last_name	AZ province_name
1	Donald	Waterfield	Ontario
2	Mickey	Baasha	Ontario
3	Jiji	Sharma	Ontario
4	Blair	Diaz	Ontario
5	Charles	Wolfe	Ontario
6	Sue	Falcon	Ontario
7	Thomas	ONeill	Ontario
8	Sonny	Beckett	Nova Scotia
9	Sister	Spitzer	Ontario
10	Cedric	Coltrane	Ontario
11	Hank	Spencer	Ontario
12	Sara	di Marco	Ontario
13	Daphne	Seabright	Ontario
14	Rick	Bennett	Ontario
15	Amy	Leela	Ontario
16	Woody	Bashir	Ontario

Text

Record

Refresh Save Cancel Export data 200 200+

Question 8

Show how many patients have a birth_date with 2010 as the birth year.

```
SELECT COUNT(*) AS total_patients FROM patients WHERE YEAR(birth_date) = 2010;
```

YEAR() extracts the year portion of a date; COUNT(*) tallies matching rows.

```
-- Q8. Show how many patients have a birth_date with 2010 as the birth year;
select count(*) as total_patients
from patients
where year (birth_date) = 2010;

-- Q9. Show first name, last name, and height of the patient with the greatest height
```

patients 1 X

select count(*) as total_patients from patients

Enter a SQL expression to filter results (use Ctrl+Space)

	total_patients
1	55

refresh Save Cancel Export data 200 1

row(s) fetched - 0.19s, on 2026-08-06 at 11:51:56

IST en Writable Smart Insert 59 : 1 [81]

Question 9

Show the first_name, last_name, and height of the patient with the greatest height.

```
SELECT first_name, last_name, height FROM patients ORDER BY height DESC LIMIT 1;
```

ORDER BY + LIMIT 1 is the standard pattern for finding the record holding an extreme value, as opposed to MAX() which only returns the number itself.

```
-- Q9. Show first name, last name, and height of the patient with the greatest height
select first_name, last_name, height
from patients
order by height DESC
limit 1;

-- another method gets only the number
select max(height)
from patients;
```

patients 1 X

select first_name, last_name, height from patients

Enter a SQL expression to filter results (use Ctrl+Space)

	AZ first_name	AZ last_name	height
1	Sam	Haruko	226

refresh Save Cancel Export data 200 1

Question 10

Show all columns for patients who have one of these patient_ids: 1, 45, 534, 879, 1000.

```
SELECT * FROM patients WHERE patient_id IN (1, 45, 534, 879, 1000);
```

IN() checks against a list of values, shorthand for multiple OR conditions.

The screenshot shows a database interface with a query editor and a results grid. The query editor contains the SQL statement: `select * from patients where patient_id`. The results grid displays the following data:

	patient_id	first_name	last_name	gender	birth_date	city	province_id
1	1	Donald	Waterfield	M	1963-02-12	Barrie	ON
2	45	Cross	Gordon	M	2009-03-20	Ancaster	ON
3	534	Don	Zatara	M	2008-01-11	Timmins	ON
4	879	Orla	Shawn	F	1967-09-24	Sarnia	ON
5	1,000	Rick	Williams	M	1975-04-13	Hamilton	ON

Question 11

Show the total number of admissions.

```
SELECT COUNT(*) AS total_admissions FROM admissions;
```

A simple row count across the entire table.

The screenshot shows a database interface with a query editor and a results grid. The query editor contains the SQL statement: `-- Q11. Show the total number of admissions`
`select count(*) as total_admissions`
`from admissions;`
`select * from admissions;`
`-- Q12. show all columns from admissions where the patient was admitted and discharged on the same date`
`select *`
`from admissions`. The results grid displays the following data:

	total_admissions
1	5,067

Question 12

Show all columns from admissions where the patient was admitted and discharged on the same day.

```
SELECT * FROM admissions WHERE admission_date = discharge_date;
```

Compares two columns from the same row directly, rather than a column against a fixed value.

admissions 1 X

select * from admissions where admission_date = discharge_date

Ctrl+click to open SQL console

	123 patient_id	admission_date	discharge_date	AZ diagnosis	123 attending_doctor
1	1	2018-09-20	2018-09-20	Ineffective Breathing Pattern R/T Fluid Accumulation	
2	9	2018-12-31	2018-12-31	Ruptured Appendicitis	
3	10	2019-02-27	2019-02-27	Lower Quadrant Pain	
4	17	2019-03-04	2019-03-04	Diabetes Mellitus	
5	28	2019-03-30	2019-03-30	Cancer Of The Stomach	
6	31	2018-09-26	2018-09-26	Cardiovascular Disease	
7	53	2018-10-24	2018-10-24	Urinary Tract Infection	
8	54	2019-04-07	2019-04-07	Hypertension	
9	70	2018-07-17	2018-07-17	Migraine	
10	78	2018-06-17	2018-06-17	Hypertension	
11	91	2018-08-30	2018-08-30	Congestive Heart Failure	
12	92	2019-01-03	2019-01-03	Osteoarthritis Knee	
13	93	2019-05-20	2019-05-20	Left Cerebral Vascular Accident	
14	110	2019-02-18	2019-02-18	Biliary Colic	
15	111	2018-08-25	2018-08-25	Subdural Hematoma	
16	113	2018-10-21	2018-10-21	2nd & 3rd Degree Burns To Right Arm / Both Legs	
17	114	2019-02-11	2019-02-11	MVA HI	

Refresh Save Cancel Export data 200 200+

Question 13

Show the total number of admissions for patient_id 579.

```
SELECT COUNT(*) AS total_admissions FROM admissions WHERE patient_id = 579;
```

COUNT scoped down with a WHERE filter to a single patient's admission history.

```
-- Q13. show the total number of admissions for patient_id 579

select count(*) as total_admissions
from admissions
where patient_id = 579;

-- Q14. Based on the cities that patients live in , show unique cities that are in province_id 'NS'

select distinct city
from patients
where province_id = 'NS';
```

admissions 1 X

select count(*) as total_admissions from admissions where patient_id = 579;

Enter a SQL expression to filter results (use Ctrl+Space)

123 total_admissions
2

Panels

Question 14

Based on the cities that patients live in, show unique cities that are in province_id 'NS'.

```
SELECT DISTINCT city FROM patients WHERE province_id = 'NS';
```

DISTINCT removes duplicate values from the result set, returning each city once regardless of how many patients live there.

The screenshot shows a database query interface. The query editor at the top contains the SQL: `select distinct city from patients where`. Below the editor, a table titled "patients 1" displays the results. The table has one column, "city", and three rows of data: "Port Hawkesbury", "Halifax", and "Inverness". The interface includes a sidebar with "Grid" and "Text" views, and a "Panels" section on the right.

	AZ city
1	Port Hawkesbury
2	Halifax
3	Inverness

Question 15

Find the first_name, last_name, and birth_date of patients who have height more than 160 and weight more than 70.

```
SELECT first_name, last_name, birth_date FROM patients WHERE height > 160 AND weight > 70;
```

AND requires both conditions to be true simultaneously for a row to qualify.

The screenshot shows a database query interface. The query editor at the top contains the SQL: `select first_name, last_name, birth_date`. Below the editor, a table displays the results. The table has three columns: "first_name", "last_name", and "birth_date". There are 16 rows of data. The interface includes a sidebar with "Grid" and "Text" views, and a "Panels" section on the right. At the bottom, there is a toolbar with buttons for "Refresh", "Save", "Cancel", and "Export data".

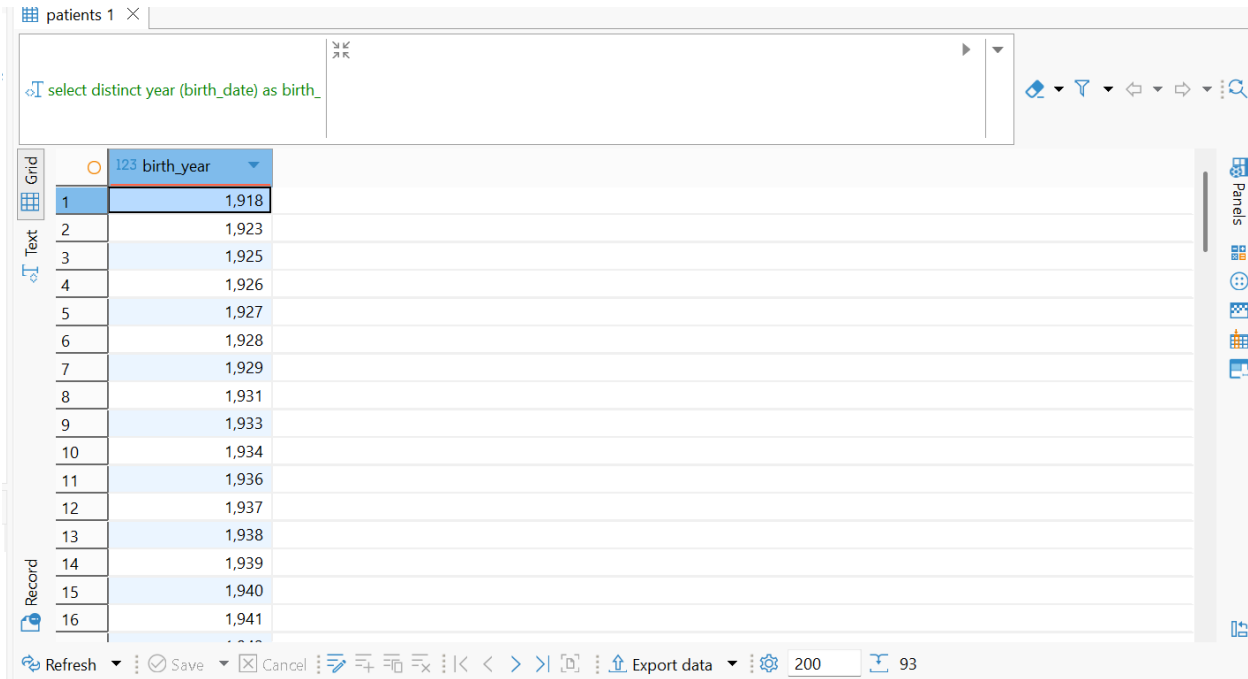
	AZ first_name	AZ last_name	birth_date
1	Alice	Donovan	1949-06-27
2	Phoebe	Bigelow	2002-03-17
3	Rose	Spellman	1957-05-31
4	Calleigh	Dean	2003-07-06
5	Vicki	Babbling	2003-08-10
6	Ernest	Clarkson	1962-11-22
7	Frank	Capone	2004-02-15
8	Beanie	Nelson	1993-06-11
9	Henry	Hernandez	1954-03-30
10	Lyan	Russo	2005-05-15
11	Thursday	Ernst	1962-11-02
12	Bernard	Karidian	2006-11-07
13	Doris	Kent	1965-09-20
14	Janice	Renault	2004-12-10
15	Sally	Smith	1951-03-18
16	Maggie	Rajasthan	1960-12-02

Question 16

Show unique birth years from patients and order them ascending.

```
SELECT DISTINCT YEAR(birth_date) AS birth_year FROM patients ORDER BY birth_year ASC;
```

Combines DISTINCT with an extracted date part and an explicit sort direction.



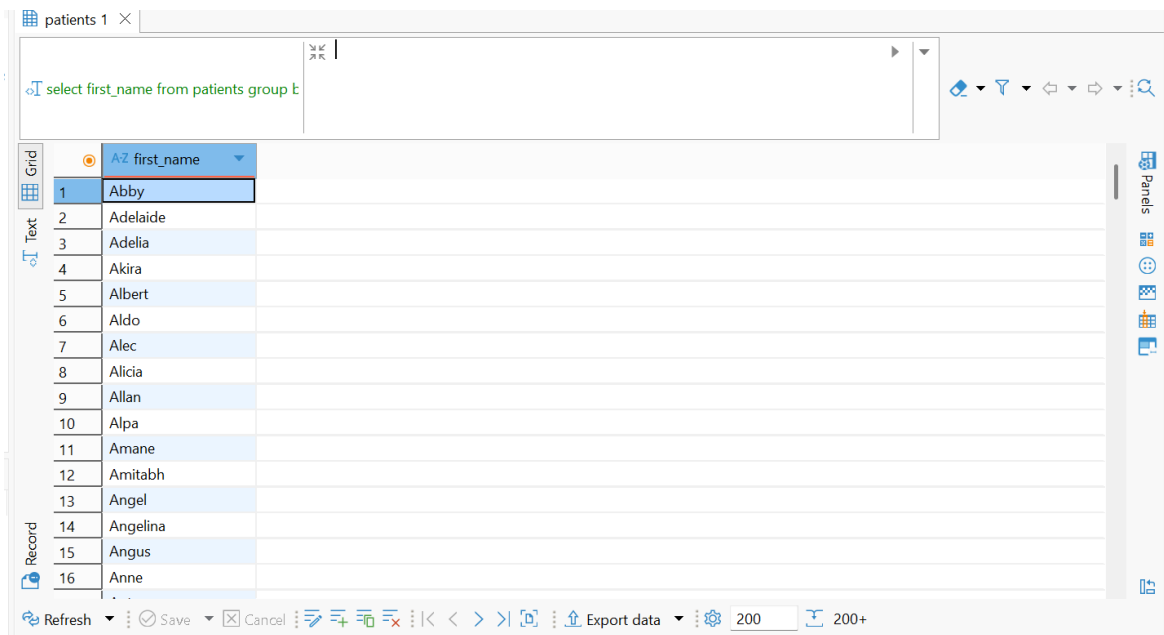
	birth_year
1	1,918
2	1,923
3	1,925
4	1,926
5	1,927
6	1,928
7	1,929
8	1,931
9	1,933
10	1,934
11	1,936
12	1,937
13	1,938
14	1,939
15	1,940
16	1,941

Question 17

Show unique first names from the patients table which only occur once in the list.

```
SELECT first_name FROM patients GROUP BY first_name HAVING COUNT(*) = 1;
```

GROUP BY clusters rows sharing the same name; COUNT(*) counts rows per cluster; HAVING filters groups (post-aggregation), unlike WHERE which filters individual rows before grouping occurs. Only groups with exactly one member survive.



	first_name
1	Abby
2	Adelaide
3	Adelia
4	Akira
5	Albert
6	Aldo
7	Alec
8	Alicia
9	Allan
10	Alpa
11	Amane
12	Amitabh
13	Angel
14	Angelina
15	Angus
16	Anne

Question 18

Show **patient_id** and **first_name** from patients where their **first_name** starts and ends with 's' and is at least 6 characters long.

```
SELECT patient_id, first_name FROM patients WHERE first_name LIKE 's%s' AND LENGTH(first_name) >= 6;
```

Combines a LIKE pattern (starts and ends with 's') with a LENGTH() check for minimum size.

	patient_id	first_name
1	496	Spiros
2	629	Spiros
3	648	Stanislaus
4	1,273	Stanislaus
5	1,789	Seamus
6	1,926	Stanislaus
7	1,996	Stanislaus
8	2,258	Spiros
9	2,378	Stanislaus
10	2,771	Stanislaus
11	3,487	Stanislaus

Question 19

Show **patient_id**, **first_name**, **last_name** from patients whose diagnosis is 'Dementia'. Primary diagnosis is stored in the admissions table.

```
SELECT p.patient_id, p.first_name, p.last_name FROM patients p JOIN admissions a ON p.patient_id = a.patient_id WHERE a.diagnosis = 'Dementia';
```

A join is required because the filter condition (diagnosis) lives in a different table than the columns being displayed.

	patient_id	first_name	last_name
1	160	Miranda	Delacour
2	178	David	Bustamonte
3	207	Matt	Celine
4	613	Jaki	Granger
5	836	Montana	Vimes
6	924	Simon	Spellman
7	1,201	Irene	Murphy
8	1,264	Jillian	Valentine
9	1,402	Kathryn	Hallow
10	1,491	Doris	McGrew
11	1,585	Alex	Cantropus
12	1,749	Alejandro	Mellie
13	1,798	Sister	Trenton
14	1,844	Jadu	Wilson
15	1,938	Anthony	Anneke
16	2,292	Jessica	Talbot

Question 20

Display every patient's first_name. Order the list by the length of each name and then alphabetically.

```
SELECT first_name FROM patients ORDER BY LENGTH(first_name), first_name;
```

Multiple ORDER BY keys apply in sequence — the second key only breaks ties left by the first.

	first_name
1	Al
2	Al
3	Al
4	Al
5	Al
6	Al
7	Al
8	Bo
9	Bo
10	Bo
11	Bo
12	Bo
13	Bo
14	Cy
15	Cy
16	Ed

Question 21

Show the total amount of male patients and the total amount of female patients in the patients table. Display the two results in the same row.

```
SELECT SUM(CASE WHEN gender = 'M' THEN 1 ELSE 0 END) AS male_count, SUM(CASE WHEN gender = 'F' THEN 1 ELSE 0 END) AS female_count FROM patients;
```

CASE WHEN acts as SQL's if/else logic. SUM() over the resulting 1s/0s produces both counts side-by-side in a single row, rather than the separate rows a plain GROUP BY gender would produce.

```
-- Q21&Q22. show total male patients and total female patients, in the same row
select
  sum(case when gender = 'M' then 1 else 0 end) as male_count,
  sum(case when gender = 'F' then 1 else 0 end) as female_count
from patients;

-- Q23. Show patient id, diagnosis form admissions- find patients admitted multiple times for the same diagnosis
select patient_id, diagnosis
from admissions
group by patient_id, diagnosis
having count(*) > 1;
```

SQL Error [1370] [42000]: execute command denied to user 'dm_team4'@'%' for routine project_medical_data_history.sum

```
select
  sum(case when gender = 'M' then 1 else 0 end) as male_count,
  sum(case when gender = 'F' then 1 else 0 end) as female_count
from patients
LIMIT 0, 200
```

Question 22

(Duplicate of Q21 on the source sheet — same query and result applies.)

-- See Q21

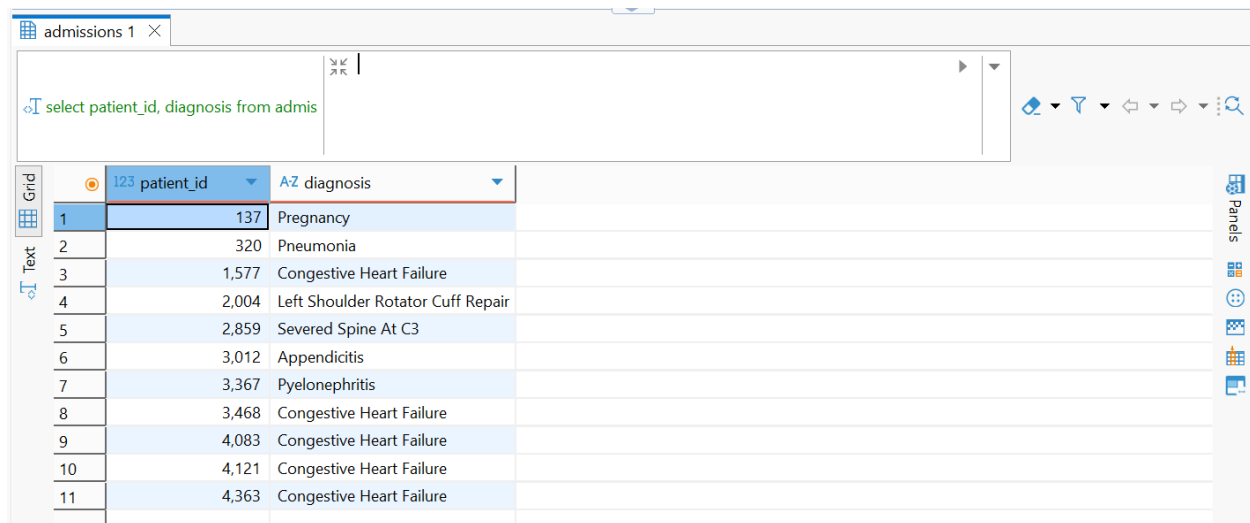
Repeated question; answered once above.

Question 23

Show patient_id, diagnosis from admissions. Find patients admitted multiple times for the same diagnosis.

```
SELECT patient_id, diagnosis FROM admissions GROUP BY patient_id, diagnosis
HAVING COUNT(*) > 1;
```

Grouping by two columns together means a group only forms when both values match, capturing "same patient, same diagnosis" rather than just repeat admissions of any kind.



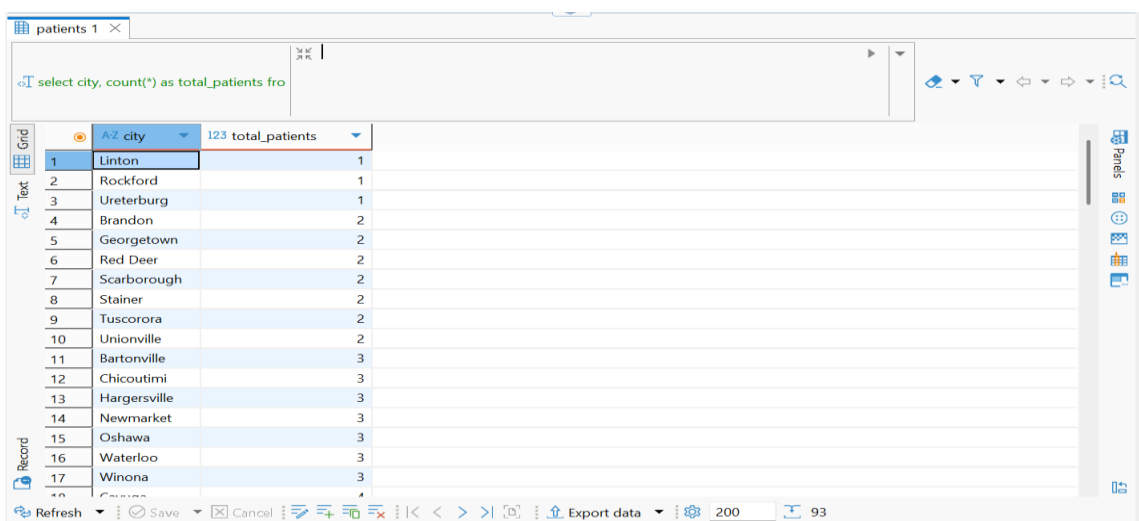
	patient_id	diagnosis
1	123	Pregnancy
2	320	Pneumonia
3	1,577	Congestive Heart Failure
4	2,004	Left Shoulder Rotator Cuff Repair
5	2,859	Severed Spine At C3
6	3,012	Appendicitis
7	3,367	Pyelonephritis
8	3,468	Congestive Heart Failure
9	4,083	Congestive Heart Failure
10	4,121	Congestive Heart Failure
11	4,363	Congestive Heart Failure

Question 24

Show the city and the total number of patients in the city. Order from most to least patients and then by city name ascending.

```
SELECT city, COUNT(*) AS total_patients FROM patients GROUP BY city ORDER BY
total_patients DESC, city ASC;
```

A two-level sort: primarily by patient count descending, with city name as an alphabetical tiebreaker only when counts are equal.



	city	total_patients
1	Linton	1
2	Rockford	1
3	Ureterburg	1
4	Brandon	2
5	Georgetown	2
6	Red Deer	2
7	Scarborough	2
8	Stainer	2
9	Tuscorora	2
10	Unionville	2
11	Bartonville	3
12	Chicoutimi	3
13	Hargersville	3
14	Newmarket	3
15	Oshawa	3
16	Waterloo	3
17	Winona	3

Question 25

Show first name, last name and role of every person that is either patient or doctor. Roles are "Patient" or "Doctor".

```
SELECT first_name, last_name, 'Patient' AS role FROM patients UNION ALL SELECT first_name, last_name, 'Doctor' AS role FROM doctors;
```

UNION ALL stacks the results of two separate SELECT statements into one combined list (as opposed to JOIN, which lines tables up side-by-side). The literal strings 'Patient'/'Doctor' label the source of each row.

The screenshot shows a database query results window. At the top, a text box contains the query: `select first_name, last_name, 'Patients' as`. Below the text box is a table with 16 rows and 4 columns. The columns are labeled 'A-Z first_name', 'A-Z last_name', and 'A-Z role'. The first column is numbered 1 through 16. The data in the table is as follows:

	A-Z first_name	A-Z last_name	A-Z role
1	Donald	Waterfield	Patients
2	Mickey	Baasha	Patients
3	Jiji	Sharma	Patients
4	Blair	Diaz	Patients
5	Charles	Wolfe	Patients
6	Sue	Falcon	Patients
7	Thomas	ONeill	Patients
8	Sonny	Beckett	Patients
9	Sister	Spitzer	Patients
10	Cedric	Coltrane	Patients
11	Hank	Spencer	Patients
12	Sara	di Marco	Patients
13	Daphne	Seabright	Patients
14	Rick	Bennett	Patients
15	Amy	Leela	Patients
16	Woody	Bashir	Patients

At the bottom of the window, there is a status bar that reads: "200 row(s) fetched - 0.406s, on 2026-08-06 at 12:52:53".

Question 26

Show all allergies ordered by popularity. Remove NULL values from the query.

```
SELECT allergies, COUNT(*) AS total FROM patients WHERE allergies IS NOT NULL GROUP BY allergies ORDER BY total DESC;
```

Filters out missing values before grouping, then ranks remaining values by frequency.

patients 1

select allergies, count(*) as total from p

	AZ allergies	123 total
1	Penicillin	1,082
2	Codeine	305
3	Sulfa	157
4	ASA	99
5	Sulfa Drugs	71
6	Peanuts	52
7	Iodine	48
8	Tylenol	42
9	Bee Stings	40
10	Valporic Acid	38
11	Tetracycline	34
12	Wheat	33
13	Peanut Butter	29
14	Micropore Tape	27
15	Hay Fever	25
16	Strawberries	25

Refresh Save Cancel Export data 200 63

63 row(s) fetched - 0.233s, on 2026-08-06 at 12:55:37

Question 27

Show all patients' first_name, last_name, and birth_date who were born in the 1970s decade. Sort starting from the earliest birth_date.

```
SELECT first_name, last_name, birth_date FROM patients WHERE birth_date BETWEEN '1970-01-01' AND '1979-12-31' ORDER BY birth_date ASC;
```

Dates compare the same way numbers do once formatted correctly (YYYY-MM-DD).

patients 1

select first_name, last_name, birth_date

	AZ first_name	AZ last_name	birth_date
1	Frances	Kobayakawa	1970-01-02
2	Sunny	Burrell	1970-01-07
3	Penelope	Beckett	1970-01-14
4	Deborah	Stewart	1970-01-14
5	Augusta	Decker	1970-01-22
6	Sookie	Brearily	1970-02-01
7	Temple	Wylie	1970-02-10
8	Deanna	Spano	1970-03-23
9	Jadu	Principal	1970-03-28
10	Betty	Stephens	1970-03-28
11	Jo	Sahid	1970-03-31
12	Santino	Hawker	1970-04-08
13	Andy	Guarnaccia	1970-04-10
14	Richard	Arroway	1970-04-13
15	Alan	Davis	1970-04-14
16	Cameron	Spellman	1970-04-21
17	Bertha	Smart	1970-05-06

Refresh Save Cancel Export data 200 200+

IST en Writable Smart Insert 201 : 1 [138]

Question 28

Display each patient's full name in a single column: last_name in uppercase first, then first_name in lowercase, separated by a comma. Order by first_name descending. Example: SMITH,jane

```
SELECT CONCAT(UPPER(last_name), ', ', LOWER(first_name)) AS full_name FROM
patients ORDER BY first_name DESC;
```

Combines UPPER(), LOWER(), and CONCAT() to build a precisely formatted output string; sorting still references the original column.

The screenshot shows a database query editor with the following SQL query:

```
select concat(Upper(last_name),',',lower
```

The results are displayed in a grid view with the following data:

	AZ full_Name
1	MILLER,zoe
2	CORBIE,ziva
3	KOBAYAKAWA,zenigata
4	OVERSTREET,zenigata
5	BENNETT,zen
6	MEPHESTO,zelda
7	MORRIS,zelda
8	THOMAS,zefram
9	FLUTE,zefram
10	TYRELL,zatanna
11	MARONEY,zatanna
12	WONG,zane
13	CHE,zane
14	DREW,zack
15	BAKSHI,zachary
16	PRESTON,yusuke

Question 29

Show the province_id(s), sum of height, where the total sum of its patients' height is greater than or equal to 7,000.

```
SELECT province_id, SUM(height) AS total_height FROM patients GROUP BY
province_id HAVING SUM(height) >= 7000;
```

HAVING works with any aggregate function, not just COUNT — here it filters groups based on a SUM threshold.

The screenshot shows a database query editor with the following SQL query:

```
-- Q29. Show province_id(s) and sum of height, where the total sum of patient height is >=7000
select province_id, sum(height) as total_height
from patients
group by province_id
having sum(height) >=7000;
```

The results are displayed in a grid view with the following data:

	AZ province_id	123 total_height
1	BC	7,720
2	NS	9,765
3	ON	678,037

3 row(s) fetched - 0.101s, on 2026-08-06 at 13:16:35

Question 30

Show the difference between the largest weight and smallest weight for patients with the last name 'Maroni'.

```
SELECT MAX(weight) - MIN(weight) AS weight_difference FROM patients WHERE last_name = 'Maroni';
```

MAX() and MIN() are aggregate functions; with no GROUP BY, the entire filtered result is treated as a single group.

```
-- Q30. Show the difference between largest and smallest weight for patients with last name "Maroni"
select max(weight) - min(weight) as weight_differences
from patients
where last_name = 'Maroni';

-- Q31. Show all the days of the month (1-31) and how many admission_dates occurred on that day.
-- sort most to least admissions
select day(admission_date) as day_of_month, count(*) as total_admissions
from admissions
```

Grid	123 weight_differences
1	71

Refresh Save Cancel Export data 200 1

1 row(s) fetched - 0.103s, on 2026-08-06 at 13:17:38

Question 31

Show all of the days of the month (1-31) and how many admission_dates occurred on that day. Sort by the day with most admissions to least.

```
SELECT DAY(admission_date) AS day_of_month, COUNT(*) AS total_admissions FROM admissions GROUP BY DAY(admission_date) ORDER BY total_admissions DESC;
```

DAY() extracts the day-of-month number. Note: this only returns days that had at least one admission; a fully exhaustive 1-31 list (including zero-count days) would require a generated reference list joined via LEFT JOIN.

```
select day(admission_date) as day_of_m
```

Grid	123 day_of_month	123 total_admissions
1	11	184
2	4	184
3	9	183
4	2	180
5	12	179
6	6	179
7	16	177
8	21	174
9	28	173
10	13	173
11	7	167
12	8	166
13	18	166
14	3	166
15	30	165
16	20	165

Column: province_id char(2)
Table: project_medical_data_history.patients
Read-only: No unique key. Row modification is not available.

Refresh Save Cancel Export data 200 31

31 row(s) fetched - 0.092s, on 2026-08-06 at 13:18:20

Question 32

patients 1 ×

select patient_id, weight, height, case w

low total

s FROM

ending up

	123 patient_id	123 weight	123 height	123 isObese
1	1	65	156	1
2	2	76	185	1
3	3	106	194	1

Results 1 ×

select floor(weight/10)*10 as weight_gr

	123 weight_group	123 total_patients
1	140	6
2	130	59
3	120	191
4	110	426
5	100	507
6	90	403
7	80	478
8	70	633
9	60	685
10	50	443
11	40	202
12	30	126
13	20	165
14	10	114
15	0	92

Column: province_id char(2)
Table: project_medical_data_history.patients
Read-only: No unique key. Row modification is not available.

Refresh Save Cancel Export data 200 15

15 row(s) fetched - 0.115s, on 2026-08-06 at 13:19:13

Question 33

Show patient_id, weight, height, isObese from the patients table. Display isObese as a boolean 0 or 1. Obese is defined as weight(kg)/height(m). Weight is in kg, height is in cm.

```
SELECT patient_id, weight, height, CASE WHEN weight / (height / 100.0) > 25  
THEN 1 ELSE 0 END AS isObese FROM patients;
```

Height is converted from cm to m before applying the BMI-style formula. CASE WHEN outputs a 0/1 boolean flag based on the threshold.

patients 1

select patient_id, weight, height, case w

	123 patient_id	123 weight	123 height	123 isObese
1	1	65	156	1
2	2	76	185	1
3	3	106	194	1
4	4	104	191	1
5	5	10	47	0
6	6	5	43	0
7	7	117	180	1
8	8	105	174	1
9	9	95	173	1
10	10	61	157	1
11	11	74	158	1
12	12	46	145	1
13	13	77	146	1
14	14	95	220	1
15	15	72	172	1
16	16	59	153	1

Refresh Save Cancel Export data 200 200+

200 row(s) fetched - 0.177s (0.002s fetch), on 2026-08-06 at 13:20:04

Question 34

Show patient_id, first_name, last_name, and attending doctor's specialty. Show only patients with a diagnosis of 'Epilepsy' where the doctor's first name is 'Lisa'.

```
SELECT p.patient_id, p.first_name, p.last_name, d.specialty FROM patients p JOIN
admissions a ON p.patient_id = a.patient_id JOIN doctors d ON a.doctor_id =
d.doctor_id WHERE a.diagnosis = 'Epilepsy' AND d.first_name = 'Lisa';
```

A three-table join chaining patients -> admissions -> doctors. Required the schema fix below (attending_doctor_id renamed to doctor_id) to align keys with the doctors table.

-- Q34. Show patientid , first name, last name, and attending doctor's speciality, on ly for patients
-- diagnosed with 'Epilepsy' where doctor's first name is 'lisa'

```
Select p.patient_id, p.first_name, p.last_name, d.specialty
from patients p
join admissions a on p.patient_id = a.patient_id
join doctors d on a.attending_doctor_id = d.doctor_id
where a.diagnosis = 'Epilepsy' and d.first_name='Lisa';

select * from doctors;
```

patients(+) 1

Select p.patient_id, p.first_name, p.last_n

Enter a SQL expression to filter results (use Ctrl+Space)

	123 patient_id	A-Z first_name	A-Z last_name	A-Z specialty
1	468	Frank	Anderson	Obstetrician/Gynecologist
2	701	Precious	Ashton	Obstetrician/Gynecologist

Refresh Save Cancel Export data 200 2

2 row(s) fetched - 0.103s (0.001s fetch), on 2026-08-06 at 13:20:48

5. Key Skills Demonstrated

Working through this project built and reinforced the following SQL and database concepts:

- Filtering data with WHERE, including text patterns (LIKE), ranges (BETWEEN), and value lists (IN)
- Handling missing data correctly with IS NULL / IS NOT NULL, and COALESCE() as a non-destructive alternative
- Aggregate functions: COUNT, SUM, MAX, MIN, and how they behave with and without GROUP BY
- GROUP BY for bucketing rows, including grouping on multiple columns and on calculated expressions
- HAVING vs. WHERE — filtering aggregated groups after grouping, versus filtering individual rows before it
- Multi-key sorting with ORDER BY, where later keys only break ties left by earlier ones
- String functions: CONCAT, UPPER, LOWER, LENGTH, and building precisely formatted output
- Date functions: YEAR(), DAY(), and comparing/filtering on date ranges
- CASE WHEN conditional logic for boolean flags and pivoting counts into columns
- INNER JOIN across two and three related tables using foreign keys
- UNION ALL for combining rows from structurally similar queries across different tables
- Schema literacy: reading an ERD, understanding primary/foreign keys, and using ALTER TABLE to rename a column for referential integrity
- Working within real-world constraints: identifying and adapting to read-only database permissions
- Practical tooling: connecting to a remote MySQL server via DBeaver, diagnosing authentication and connection errors

6. Conclusion

This project provided hands-on practice translating plain-language business questions into precise SQL queries against a real relational schema, while also navigating practical realities like database connection setup, authentication issues, and access permission constraints. The combination of query-writing and infrastructure troubleshooting reflects the kind of end-to-end problem solving expected in a data analytics role.